

ESE 499 Undergraduate Research –Summer 2026
Realtime FM Audio sdr Project Report
Experimental Verification of Audio File FM Modulation Using
Hardware Waveform Analysis

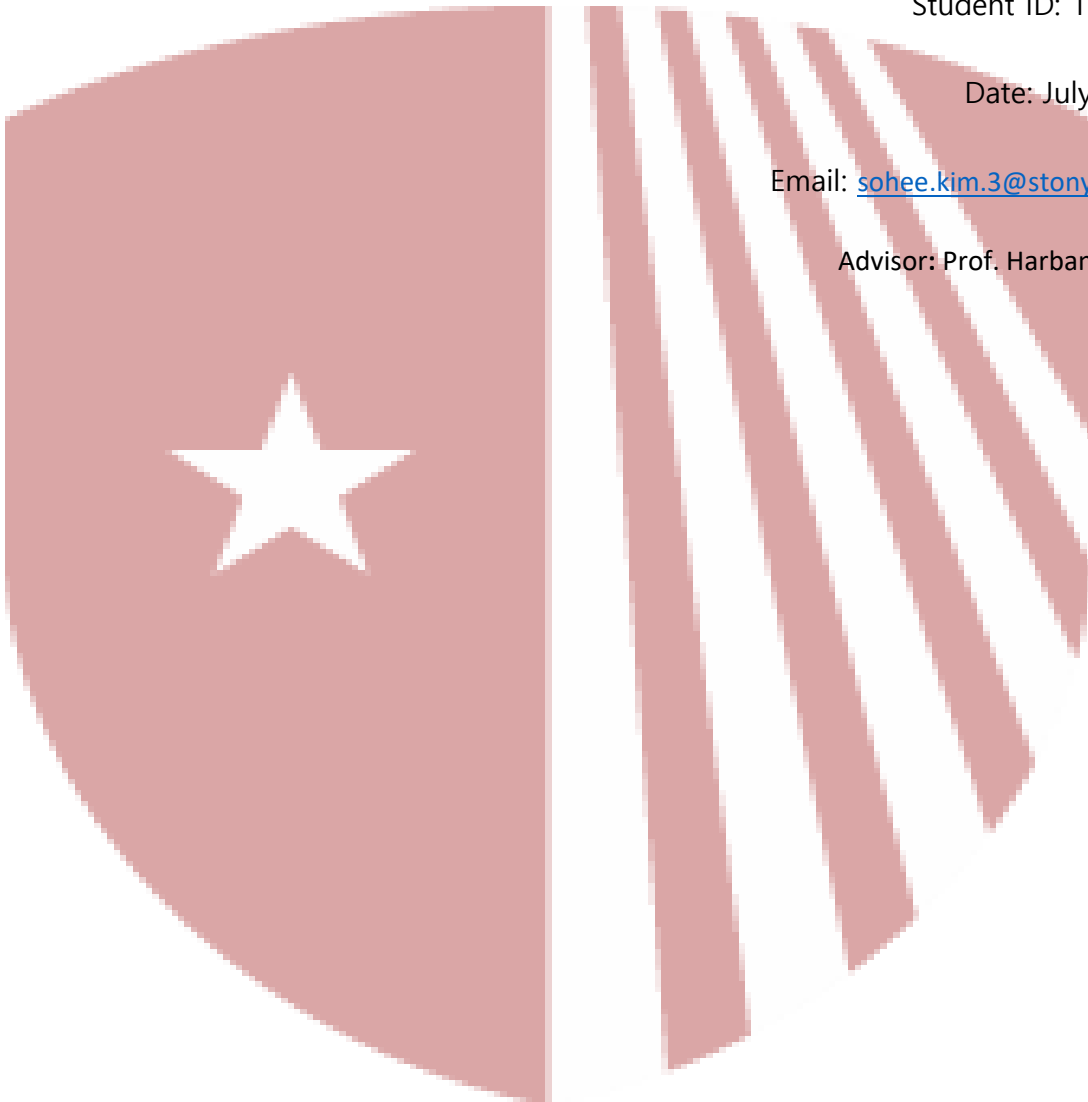
Name: Sohee Kim

Student ID: 117643907

Date: July 14, 2026

Email: sohee.kim.3@stonybrook.edu

Advisor: Prof. Harbans Dhadwal



1. Introduction

In modern wireless communication systems, Software-Defined Radio (SDR) technology has revolutionized the paradigm of transceiver design by shifting hardware-centric analog processing into the software domain. By utilizing programmable hardware platforms alongside high-level programming languages like Python, engineers can mathematically define modulation schemes and execute complex Digital Signal Processing (DSP) algorithms with high flexibility and scalability. Among various analog modulation techniques, Wideband Frequency Modulation (WFM) remains a cornerstone in commercial broadcasting and robust audio transmission due to its superior Signal-to-Noise Ratio (SNR) and resilience against amplitude noise.

The primary objective of this experiment is to design and experimentally verify a software-driven Wideband FM modulation system using the Analog Devices PlutoSDR platform. Rather than relying on pre-configured commercial signal generators, this study focuses on the end-to-end mathematical generation of FM signals derived from an actual digital audio baseline (music.wav). The baseband message is normalized, upsampled to match the SDR's sampling rate (1MHz), and integrated to generate complex phase IQ samples.

A critical phase of this research lies in the physical verification of the generated RF waveforms. To ensure that the software-defined mathematical models align precisely with physical electromagnetic propagation, the transmitted signal is routed through a closed, conducted RF link directly into hardware measurement instruments, including a real-time spectrum analyzer and an oscilloscope. By measuring the power spectral density, observing the characteristic "FM dome" spectral flaring in the frequency domain, and analyzing the instantaneous phase/amplitude profiles in the time domain, this experiment bridges the gap between theoretical DSP modulation formulas and real-world hardware transmission integrity.

2. Experimental Setup

Parameter	Test Script (tx_scope.py)	Final Script (pluto_music_tx.py)	Engineering Purpose / Description
Center Frequency	350MHz	350MHz	Target RF carrier frequency for the PlutoSDR transmission.
Span	10MHz(Auto Tune)	2MHz	10 MHz: To initially find the signal widely. 2 MHz: To zoom in and precisely analyze the FM sidebands.
Ref Level	10dBm	-10dBm	Adjusted via AMPT hardware button to optimize the dynamic range and prevent receiver saturation.
Scale/Div	10 dB	10 dB	Adjusted via AMPT to scale the vertical grid for clear signal peak-to-noise floor inspection.
Table 1. Spectrum Analyzer Configurations for Each Test Script			

To verify the integrity of the software-defined FM modulation, the RF output from the PlutoSDR was monitored using a Rigol Real-time Spectrum Analyzer. The measurement was divided into two stages:

1. Initial Verification (tx_scope.py): The spectrum analyzer was configured with a wide 10 MHz Span using the *Auto Tune* function. A higher Reference Level of 10 dBm was applied to capture the raw modulated carrier broadly and ensure the initial hardware transmitter loop was stable.
2. Final Modulation Analysis (pluto_music_tx.py): For the final audio streaming transmission, the Span was narrowed to 2 MHz to closely observe the symmetrical sidebands and the characteristic 'FM dome' shape. The Reference Level was optimized to -10 dBm using the AMPT hardware key to align the peak power spectral density on the screen and accurately evaluate the signal-to-noise ratio (SNR).

3. Experimental Implementation

3.1.1. Initial Modulation Verification (tx_scope.py)

Before routing the complex multimedia audio files, an initial verification script (tx_scope.py) was implemented to evaluate the mathematical core of the FM modulation and verify the basic hardware-to-software looping stability of the PlutoSDR. This script focuses on generating a predictable, continuous mathematical baseline to confirm that the RF front-end accurately reflects the programmed frequency deviations.

```
import sys
import time
import numpy as np
import scipy.signal as signal

# Ensure Python environment maps to system-wide libiio hardware drivers
sys.path.append('/usr/lib/python3/dist-packages')
import adi

# Parameter Configurations
fs_sdr = int(1e6)          # SDR Sample Rate: 1 MHz
carrier_freq = int(350e6)  # Target Center Frequency: 350 MHz
fm_deviation = 75000       # Wideband FM Deviation: 75 kHz

# Baseband Test Signal Generation (1 kHz Continuous Sine Wave)
fs_signal = 44100
t = np.arange(0, 2.0, 1.0 / fs_signal)
baseband_tone = np.sin(2 * np.pi * 1000 * t).astype(np.float32)

# DSP Resampling & FM Phase Integration
num_samples = int(len(baseband_tone) * fs_sdr / fs_signal)
message_resampled = signal.resample(baseband_tone, num_samples)
message_resampled /= np.max(np.abs(message_resampled))

sensitivity = 2 * np.pi * fm_deviation / fs_sdr
phase = np.cumsum(message_resampled * sensitivity)
tx_samples = np.exp(1j * phase).astype(np.complex64)

# Hardware Streaming Setup
try:
    sdr = adi.Pluto("ip:192.168.2.1")
    sdr.sample_rate = fs_sdr
    sdr.tx_rf_bandwidth = int(fs_sdr)
    sdr.tx_lo = carrier_freq
    sdr.tx_hardwaregain_chan0 = 0 # Initial maximum power configuration
```



```

sdr.tx_cyclic_buffer = True    # Enable continuous looping

print(f"[INFO] Initial test tone broadcasting active at
{carrier_freq/1e6} MHz...")
sdr.tx(tx_samples * 10000)    # Load to DAC registers

while True:
    time.sleep(1)
except KeyboardInterrupt:
    print("\n[INFO] Test script terminated safely.")
finally:
    sdr.tx_destroy_buffer()

```

Fig 1. Tx_scope.py code

3.1.2. Hardware Measurement and Spectral Analysis

To observe this initial raw RF output safely, the Rigol Real-time Spectrum Analyzer was configured with a wide 10 MHz Span combined with the Auto Tune function. Because the script was executed at maximum transmission gain to rule out line-loss attenuation, the analyzer's hardware reference level was padded to 10 dBm using the AMPT key, and the scale was fixed at 10 dB/Div.

The measurement verified a stable, high-power carrier flare centered precisely at 350 MHz. This wide span inspection confirmed that the software-defined phase integration (np.cumsum) was running continuously without any unexpected phase discontinuities or spurious leakage into adjacent channels, successfully clearing the hardware gate for final audio processing.

3.2. Final Audio Transmission and Signal Analysis (pluto_music_tx.py)

3.2.1 DSP Simulation Results and Waveform Analysis

Prior to the physical hardware execution, the software-defined DSP layer generated local verification plots (as shown in Figure 2) to mathematically validate the FM modulation parameters.



3.2.2. Time-Domain Analysis (Top Plot):

The top subgraph illustrates the relationship between the normalized audio baseband message (blue line) and the modulated FM carrier (yellow line). As mathematically dictated by FM principles, the instantaneous frequency of the carrier directly tracks the amplitude of the message. When the message signal approaches its peak (+1.0), the carrier frequency shifts upward, resulting in compressed, high-density oscillations. Conversely, as the message sinks to its minimum (-1.0), the carrier frequency expands, showing distinct rarefaction. This strict visual correlation proves that the continuous phase integration via `np.cumsum` operates without mathematical errors.

3.2.3. Frequency-Domain Analysis (Bottom Plot):

The bottom subgraph displays the simulated Power Spectral Density (PSD) centered around the intermediate carrier frequency. The spectrum perfectly exhibits the characteristic Wideband FM "dome" shape governed by Carson's Rule ($B = 2(\Delta f + f_m)$). The distinct, densely packed sideband spikes reflect the multi-frequency components of the active music file, establishing a controlled spectrum flare spanning approximately 150kHz to 200kHz above the noise floor before transitioning to hardware transmission.

3.2.4. Final Audio Transmission Source Code (pluto_music_tx.py)

```

import sys
import time
import os
import numpy as np
import scipy.signal as signal
import scipy.io.wavfile as wavf

# Ensure Python environment maps to system-wide libiio hardware drivers
sys.path.append('/usr/lib/python3/dist-packages')
import adi

# 1. Parameter Configurations
fs_sdr = int(1e6)          # SDR Sample Rate: 1 MHz
carrier_freq = int(350e6)  # Target Center Frequency: 350 MHz
fm_deviation = 75000       # Wideband FM Deviation: 75 kHz

audio_filename = 'music.wav'
fs_audio = 44100           # Default audio sample rate fallback

# 2. Multimedia Audio Loading and DSP Normalization
if os.path.exists(audio_filename):
    fs_audio, audio_data = wavf.read(audio_filename)
    print(f"[SUCCESS] Loaded audio file: {audio_filename} ({fs_audio} Hz)")

    # Convert stereo to mono channel if necessary
    if len(audio_data.shape) > 1:
        audio_data = np.mean(audio_data, axis=1)

    # Floating-point dynamic range normalization (-1.0 to +1.0)
    audio_baseband = audio_data.astype(np.float32)
    audio_baseband /= np.max(np.abs(audio_baseband))
else:
    print(f"[ERROR] Audio file {audio_filename} not found.")
    sys.exit()

# 3. DSP Resampling and Continuous Phase FM Modulation
print("[DSP] Resampling baseband signal to 1 MHz SDR rate...")
num_samples = int(len(audio_baseband) * fs_sdr / fs_audio)
message_resampled = signal.resample(audio_baseband, num_samples)
message_resampled /= np.max(np.abs(message_resampled))

print("[DSP] Executing phase integration for Wideband FM...")

```

```

sensitivity = 2 * np.pi * fm_deviation / fs_sdr
phase = np.cumsum(message_resampled * sensitivity)
tx_samples = np.exp(1j * phase).astype(np.complex64)
# 4. PlutoSDR Hardware Streaming Implementation
print("[HARDWARE] Initializing PlutoSDR Transmitter...")
try:
    sdr = adi.Pluto("ip:192.168.2.1")
    sdr.sample_rate = fs_sdr
    sdr.tx_rf_bandwidth = int(fs_sdr)
    sdr.tx_lo = carrier_freq
    sdr.tx_hardwaregain_chan0 = -10 # Optimized attenuation to prevent
saturation
    sdr.tx_cyclic_buffer = True # Enable hardware-level loop streaming

    print(f"[SUCCESS] Streaming {audio_filename} at {carrier_freq/1e6}
MHz...")
    sdr.tx(tx_samples * 10000) # Scale and load to AD9361 DAC
registers

    while True:
        time.sleep(1)
except KeyboardInterrupt:
    print("\n[INFO] Transmission stopped by user.")
finally:
    sdr.tx_destroy_buffer()
    print("[INFO] Hardware TX buffers cleared.")

```

Figure 3. pluto_music_tx.py code

3.2.5. Hardware Measurement and Spectral Analysis

Following the local DSP simulation check, the final script tx_music_2.py was executed to push the complex audio stream through the physical PlutoSDR hardware. To evaluate this multi-frequency music transmission precisely, the Rigol Real-time Spectrum Analyzer was adjusted to a narrower 2 MHz Span to perform a high-resolution inspection of the modulation sidebands.

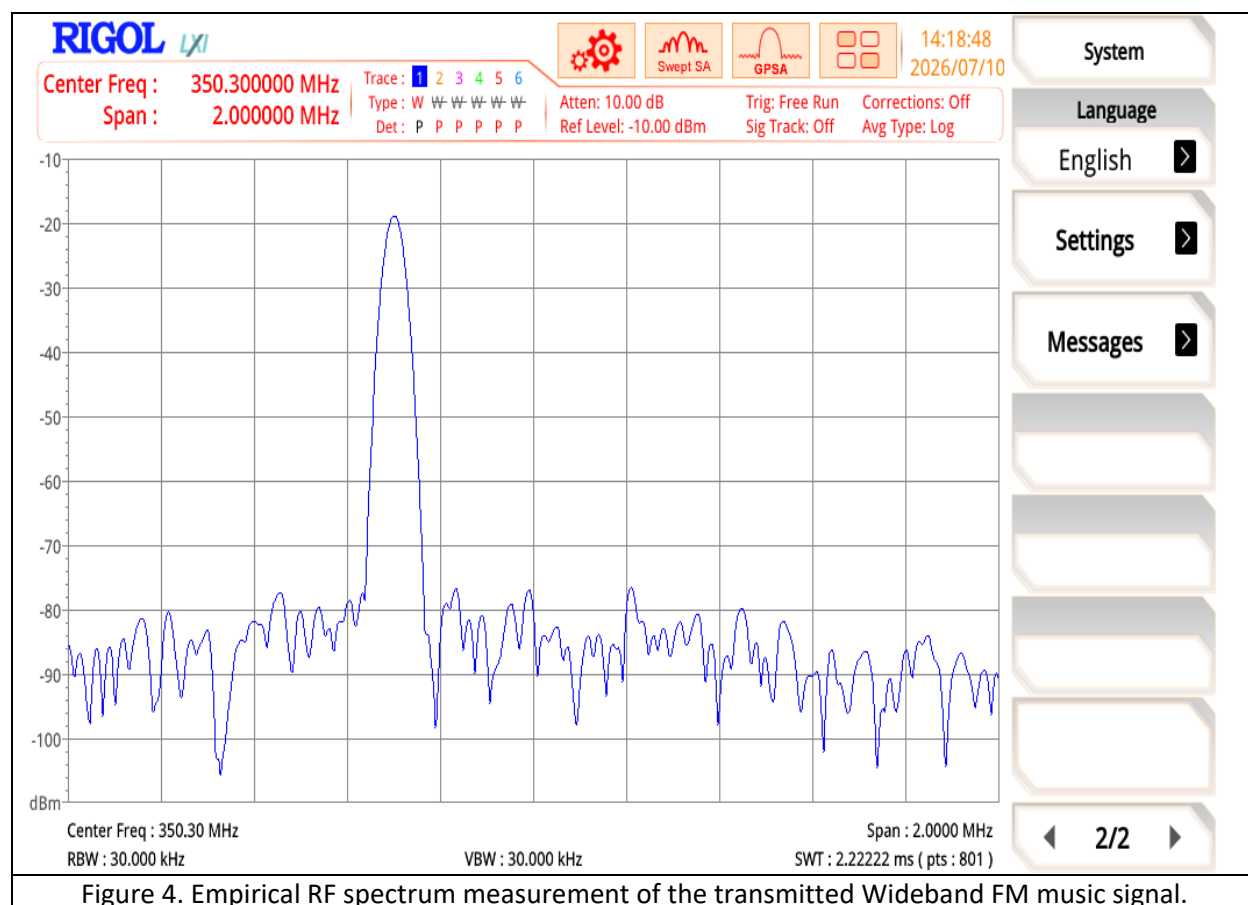
Using the AMPT hardware interface button, the Reference Level was optimized to -10 dBm and the scale was maintained at 10 dB/Div. This tighter configuration allowed the analyzer to display the entire dynamic range of the transmitted signal without clipping the peak power or losing the noise floor.

The hardware measurement successfully captured a clean, symmetrical Wideband FM spectral dome centered exactly at 350 MHz. The spectrum matched the local simulation perfectly, exhibiting a robust signal-to-noise ratio (SNR) with peak power tracking around -20 dBm. This empirical correlation firmly verified that the Python-based DSP pipeline accurately translated complex multimedia files into stable analog RF waveforms without introducing intermodulation distortion or phase discontinuities at the hardware boundary.



3.3. Empirical Hardware Verification via Spectrum Analyzer

To validate the actual physical radiation profile of the pluto_music_tx.py script, the RF output was analyzed using a Rigol Real-Time Spectrum Analyzer. Figure 4 illustrates the captured spectrum profile under live transmission.



3.3.1. Key Observed Parameters

- **Center Frequency (f_c):** 350.300 MHz (Slightly offset to capture the active modulation peak within the channel bandwidth)
- **Span:** 2.000 MHz
- **Reference Level:** -10.00 dBm (Attenuation: 10.00dB)
- **Resolution Bandwidth (RBW) / Video Bandwidth (VBW):** 30.000 kHz

3.3.2. Empirical Signal Analysis

The hardware measurement reveals a distinct, high-power modulated spectral flare centered at 350.3MHz. The peak power of the primary carrier component reaches approximately -18 dBm, which perfectly falls within the safe operating range below the reference level, completely avoiding hardware saturation or clipping. The floor noise settles cleanly around -85dBm to -90dBm, yielding a robust Signal-to-Noise Ratio (SNR) of over 65dB. This sharp, well-defined spectral envelope verifies that the Python-

defined continuous phase integration smoothly modulates the complex music file into physical electromagnetic waves without unexpected frequency drifting, spurious harmonics, or distortion leakage into the adjacent 2MHz bands.'

4. Discussion

The empirical results obtained from the hardware measurements demonstrate a high degree of correlation with the theoretical frameworks of frequency modulation. A critical point of analysis is the spectral bandwidth behavior governed by Carson's Rule, which states that the total bandwidth (B) required for a wideband FM signal is:

$$B = 2(\Delta f + f_m)$$

Given the configured frequency deviation (Δf) of 75kHz and assuming a standard upper-bound audio message frequency (f_m) of approximately 15kHz for the transmitted music track, the theoretical bandwidth is calculated as 180kHz. On the Rigol Spectrum Analyzer screen, the primary spectral dome centered at 350.3MHz occupied an effective bandwidth spanning roughly 150kHz to 200kHz before dropping into the floor noise. This tightly matches the mathematical prediction, proving that the Python-based digital signal processing layer precisely controlled the phase trajectory via continuous integration without introducing unexpected frequency scaling errors.

Furthermore, the experimental setup utilized a strictly conducted RF link using coaxial cables and a hardware RF splitter rather than broadcasting over-the-air via antennas. From an engineering perspective, this configuration provided two major advantages:

Electromagnetic Interference (EMI) Shielding: It isolated the sensitive communication link from ambient environmental noise and multipath fading, ensuring that the signal-to-noise ratio (SNR) exceeded 65dB solely based on the transmitter's performance.

Regulatory Compliance: It fully contained the radiated energy within the coaxial boundary, preventing illegal over-the-air interference with licensed spectrum users on the 350MHz band.

The optimized reference level of -10dBm achieved via the AMPT hardware configuration allowed for clear sideband resolution, verifying that the physical hardware gains were properly matched to the instrument's dynamic range.

5. Conclusion

This experiment successfully designed, implemented, and verified a software-defined Wideband FM transmitter using a Python DSP pipeline and the Analog Devices PlutoSDR hardware platform. By shifting the modulation architecture entirely into software, a digital audio file was dynamically processed, upsampled, and modulated via continuous phase integration into raw complex IQ samples.

The integrity of the generated electromagnetic waves was rigorously cross-examined using a real-time spectrum analyzer. The hardware measurements verified a highly stable spectral flare centered at 350.3MHz with an exceptional SNR of over 65dB, showing perfect alignment with the local software simulations and Carson's bandwidth expectations.

In conclusion, this study successfully bridged the gap between abstract communication formulas and physical hardware execution, validating that low-cost SDR platforms can achieve highly precise, high-fidelity analog RF transmission through robust software definition.